

PyVSC

Constraints & Randomization

SyoSil, 2026

Contents

- Intro on Python and Randomization
- PyVCS
- Demos

Randomization & Verification Ecosystem (Python)

- **Constrained-Random (hardware-style)**
 - **PyVSC (vsc)**
 - rand fields + constraint solver, dist/weight, inline randomize_with, solve order, soft, unique; great for RTL CRV. Pairs well with **cocoTB** and **PyUVM**.
 - Constrainedrandom
 - Cocotb-coverage.crv
 - And a few others
- **Testbench frameworks**
 - **PyUVM** — UVM in Python on top of cocotb (components, phases, sequences, factory). Uses Python for RNG
 - **cocoTB** — coroutine testbench for HDL sims; integrates with Python libs (PyVSC/Hypothesis/NumPy). Not a randomization library by itself.
- **Property-based generators**
 - **Hypothesis** — data strategies + *shrinking* for minimal counter-examples. Great for protocol/state machines at the Python boundary; no cross-field solver like PyVSC, but you can compose strategies to emulate constraints.
- **General RNG / Distributions**
 - **random (stdlib)** — simple PRNG + seeding; fine for ad-hoc randomness, no constraints.
 - **NumPy default_rng** — fast vectorized distributions (normal, Poisson, etc.), reproducible via seeds; still no constraint solver logic.
 - **secrets (stdlib)** — cryptographically strong randomness; usually unnecessary for verification stimulus (slower).
 - **SciPy stats** — rich statistical distributions/samplers; heavier dependency.

Randomization & Verification Ecosystem (Python)

- **Typical combos**
 - ***PyVSC + PyUVM***
 - CRV sequences inside UVM-style environment
 - Easy to scale and reuse.
 - ***PyVSC + CocoTB***
 - Lightweight CRV driving HDL directly via coroutines.
 - ***Hypothesis + CocoTB***
 - Property-based edge discovery
 - Shrinking for debug
 - Add cocotb-coverage to measure progress.

Constrained-Random Verification (CRV) with PyVSC

- **Goal today:** use PyVSC to create high-quality *randomized* stimulus for RTL verification.
- Why CRV? State-space of real designs explodes
 - Directed tests miss corners.
- PyVSC gives SystemVerilog-style
 - *rand fields and constraints* in Python.
- We'll focus on **randomization** (Coverage comes later).
- Fits well with cocotb/pytest/simulation flows and quick prototyping.

PyVSC Abstract Model (SV → Python)

- **Random Objects (`@vsc.randobj`)**
 - Class whose fields can be randomized
- **Random types**
 - `rand_uint8_t`, `rand_bit_t(width)`, `rand_int_t(width)`,
enums and lists
- **Constraints (`@vsc.constraint`)**
 - Methods with `@vsc.constraint`; use normal math/logic ops
- **Randomize**
 - `obj.randomize()` or inline/temporary with `randomize_with`
- **Distributions**
 - bias values with `vsc.dist([...])` and `vsc.weight(...)`.

Hello, World!

```
import vsc

@vsc.randobj
class Packet:
    def __init__(self):
        self.addr      = vsc.rand_uint8_t() # 0..255
        self.length    = vsc.rand_uint8_t() # 0..255
        self.is_read   = vsc.rand_bit_t(1)  # 0/1

    @vsc.constraint
    def legal(self):
        self.length > 0      # No zero-length
        self.addr % 4 == 0  # 4-byte alignment

p = Packet()

for _ in range(3):
    p.randomize()
    print(int(p.addr), int(p.length), int(p.is_read))
```

Constraint Basics (Ranges, Sets, Relations)

- Use Python operators
 - `<`, `<=`, `==`, `!=`, `%`, `>>`, `<<`, & etc.
- Constrain to sets/ranges
 - `vsc.rangelist()`
 - `vsc.rng(lo,hi)`.
- **`inside()` / `not_inside()`**
methods are equivalents for set membership.

```
@vsc.randobj
class Arith:
    def __init__(self):
        self.a = vsc.rand_uint8_t()
        self.b = vsc.rand_uint8_t()

    @vsc.constraint
    def c_rel(self):
        self.a < self.b
        self.a in vsc.rangelist(1, 2, vsc.rng(4, 8))
        self.b.inside(vsc.rangelist(vsc.rng(2,20)))
```

Conditional Constraints (If/Else, Implies)

- Capture protocol-style conditions and structural guards
 - **vsc.if_then**
 - **vsc.else_if**
 - **vsc.else_then**
 - **vsc.implies**

```
@vsc.randobj
class Cmd:
    def __init__(self):
        self.op = vsc.rand_uint8_t()
        self.size = vsc.rand_uint8_t()

    ...
    @vsc.constraint
    def c_ops(self):
        with vsc.if_then(self.op == 0): # NOP
            self.size == 0
        with vsc.else_if(self.op == 1): # READ
            self.size in vsc.rangelist(1, 4, 8)
        with vsc.else_then:
            self.size in vsc.rangelist(vsc.rng(1,16))

    @vsc.constraint
    def c_implies(self):
        with vsc.implies(self.op == 2): # WRITE→size>=1
            self.size >= 1
```

Biasing with Distributions (dist/weight)

- Model realistic traffic patterns (hot addresses, common sizes)
- Weights apply only to listed values/ranges; others are excluded

```
@vsc.randobj
class Biased:
    def __init__(self):
        self.length = vsc.rand_uint8_t()

    @vsc.constraint
    def c_len_dist(self):
        vsc.dist(self.length, [
            vsc.weight(1, 50),      # single-beat favored
            vsc.weight(2, 20),
            vsc.weight((3,8), 30)]) # 3..8 less frequent
```

Inline Constraints with `randomize_with`

- Add **temporary** constraints without changing the class
- Useful for scenario sequencing and corner injections

```
pkt = Packet()

# Ordinary randomization respects class constraints
pkt.randomize()

# One-off: force aligned 64B burst this time
with pkt.randomize_with() as it:
    it.length == 64
    it.addr % 64 == 0

# Global variant on scalars/objects
a = vsc.rand_uint8_t()
b = vsc.rand_uint8_t()
with vsc.randomize_with(a, b):
    a < b
```

Lists, foreach, and structural constraints

- Randomize arrays/packet fields with **vsc.foreach** over list types
- For variable-size lists use **randsz_list_t** and constrain **.size**

```
@vsc.randobj
class Burst:
    def __init__(self):
        self.nbeats = vsc.rand_uint8_t()
        # Fixed size 16
        self.data = vsc.rand_list_t(vsc.uint8_t(), 16)

    @vsc.constraint
    def c_shape(self):
        self.nbeats in vsc.rangelist((1,16))
        with vsc.foreach(self.data, idx=True) as i:
            # Only first nbeats are used; unused must be zero
            vsc.implies(i >= self.nbeats, self.data[i] == 0)
```

Soft Constraints, Solve Order, Uniqueness

- **`vsc.soft(expr)`**
 - defaults that can be overridden inline or by other constraints
- **`vsc.solve_order(a, b)`**
 - influence solver order to shape distributions
- **`vsc.unique([vars])`**
 - all in list must be different.

```
@vsc.randobj
class Fancy:
    def __init__(self):
        self.a = vsc.rand_uint8_t()
        self.b = vsc.rand_uint8_t()
        self.c = vsc.rand_uint8_t()

    @vsc.constraint
    def c_misc(self):
        # Default a=0 unless something conflicts
        vsc.soft(self.a == 0)

        # Solve a before b
        vsc.solve_order(self.a, self.b)

        # a, b and c should be unique
        vsc.unique([self.a, self.b, self.c])
```

Reproducibility & Seeding (Random Stability)

- Each @randobj has its own random state
- Seed for reproducible debug; re-use seed for identical sequences

```
from vsc.methods import RandState

it = Packet()
seed = RandState.mkFromSeed(1234, "exp1")
it.set_randstate(seed)

seq1=[]; seq2=[]
for _ in range(5):
    it.randomize()
    seq1.append((int(it.addr), int(it.length)))

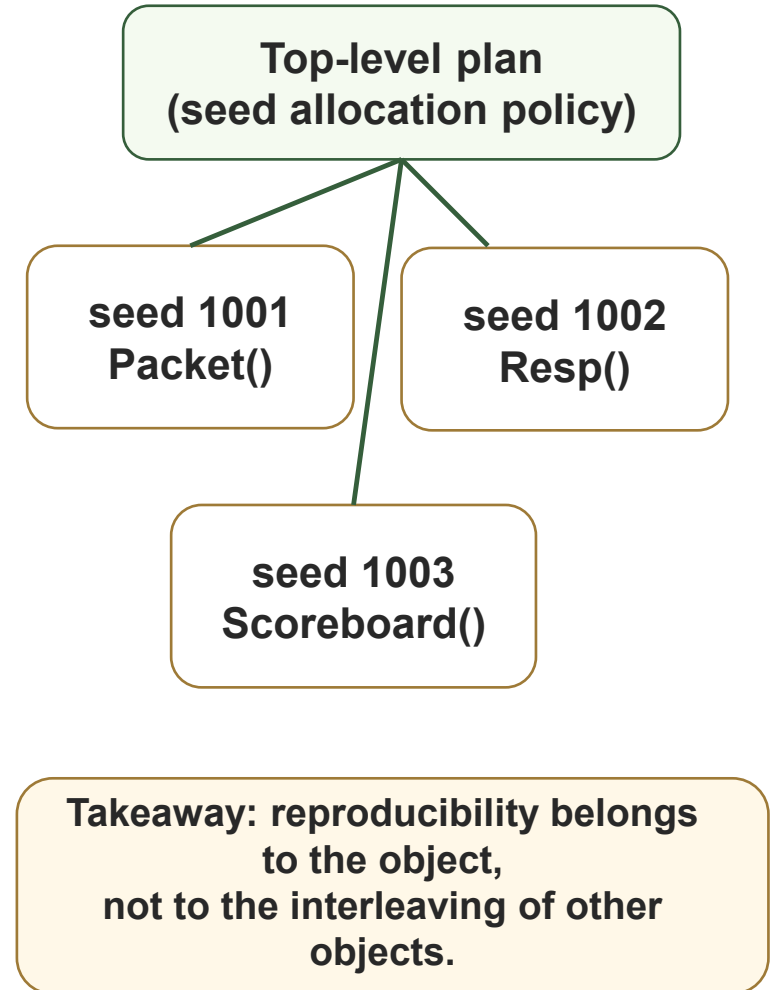
# Reset to same seed → same stream
it.set_randstate(seed)
for _ in range(5):
    it.randomize()
    seq2.append((int(it.addr), int(it.length)))

assert seq1 == seq2
```

Manual Seeding in OOP Testbenches

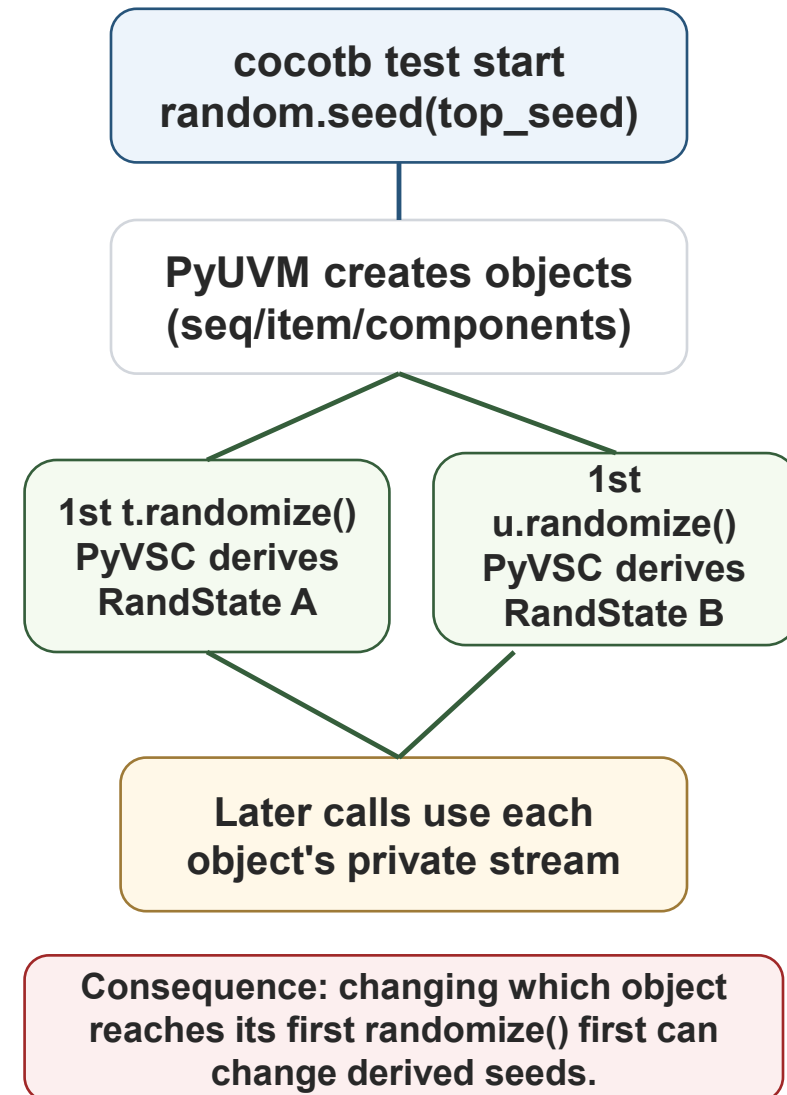
- **Idea**

- Each object owns its own RNG stream.
- You explicitly seed the object when it is created.
- Later randomization in other objects does not perturb this object's stream.
- This gives object-level random stability and easier debug/replay.
- Typical implementation: Python `random.Random(seed)` or PyVSC `set_randstate(...)`.
- Important: this is a manual technique — not the default cocotb/PyUVM seeding flow.



How it works with cocotb + PyUVM + PyVSC

- cocoTB seeds Python's global RNG once per test (for example via `COCOTB_RANDOM_SEED`).
- PyUVM provides the OOP/UVM structure: tests, components, sequences, and sequence items.
- A PyVSC @randobj automatically gets its own RandState from Python's global RNG.
- That RandState is then private to the object, so repeated `randomize()` calls advance that object's own stream.
- Because the object seed is derived from the global stream, first-randomize order matters.
- If you need stronger object-level stability, manually assign randstate with `set_randstate(...)`.



Example: random wait causes instability

Objects

```
import random, cocotb, vsc
from cocotb.triggers import Timer, Combine
from pyuvvm import uvm_sequence_item
```

```
@vsc.randobj
class Txn(uvm_sequence_item):
    def __init__(self,
                  name="txn"):
        super().__init__(name)
        self.addr = vsc.rand_uint8_t()
        self.length = vsc.rand_uint8_t()
```

```
t0 = Txn('t0')
t1 = Txn('t1')
t2 = Txn('t2')
```

3 parallel processes

```
async def run_obj(t, tag):
    for i in range(3):
        d = random.randint(1, 5)
        await Timer(d, unit="ns")
        t.randomize()
        print(tag, i, d,
              int(t.addr),
              int(t.length))

await Combine(
    cocotb.start_soon(run_obj(t0, 't0')),
    cocotb.start_soon(run_obj(t1, 't1')),
    cocotb.start_soon(run_obj(t2, 't2')))
```

Interpretation

```
# Consequence
# whichever task reaches
# its FIRST randomize()
# first gets derived stream A
# next one gets B
# next one gets C
#
# random 1..5 ns waits
# can reorder that event
# and change object numbers
```

The random wait is BEFORE t.randomize(), so task scheduling can change which object gets stream A / B / C.

Therefore, t0 / t1 / t2 are individually stable after their first randomize(), but the object-to-stream mapping is unstable.

This is a good demonstration of random instability caused by first-randomize race ordering.

PyVSC Inside a PyUVM Sequence

```
import vsc
from pyuvm import uvm_sequence,
uvm_sequence_item

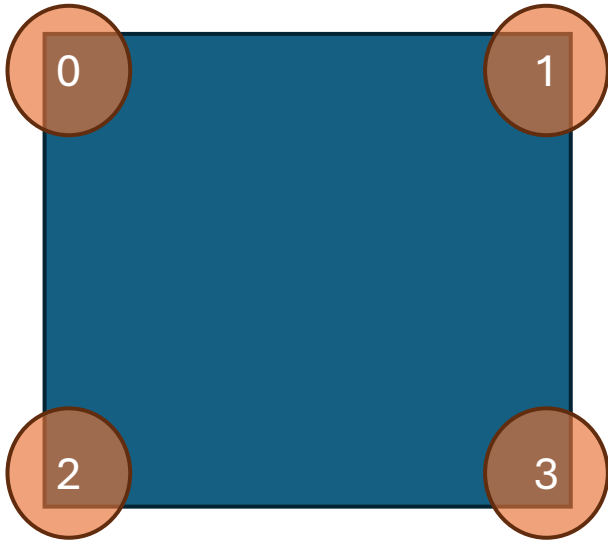
@vsc.randobj
class Txn(uvm_sequence_item):
    def __init__(self):
        self.addr      = vsc.rand_uint8_t()
        self.len       = vsc.rand_uint8_t()
        self.is_read   = vsc.rand_bit_t(1)

    @vsc.constraint
    def legal(self):
        self.len > 0
        self.addr % 4 == 0
```

```
class MySeq(uvm_sequence):
    def body(self):
        for _ in range(100):
            t = Txn()
            self.start_item(t)
            t.randomize()
            self.finish_item(t)
```

What would a Typical Regression Cover

- Typically
 - single random test
 - Then one covering the 4 corners of the state space



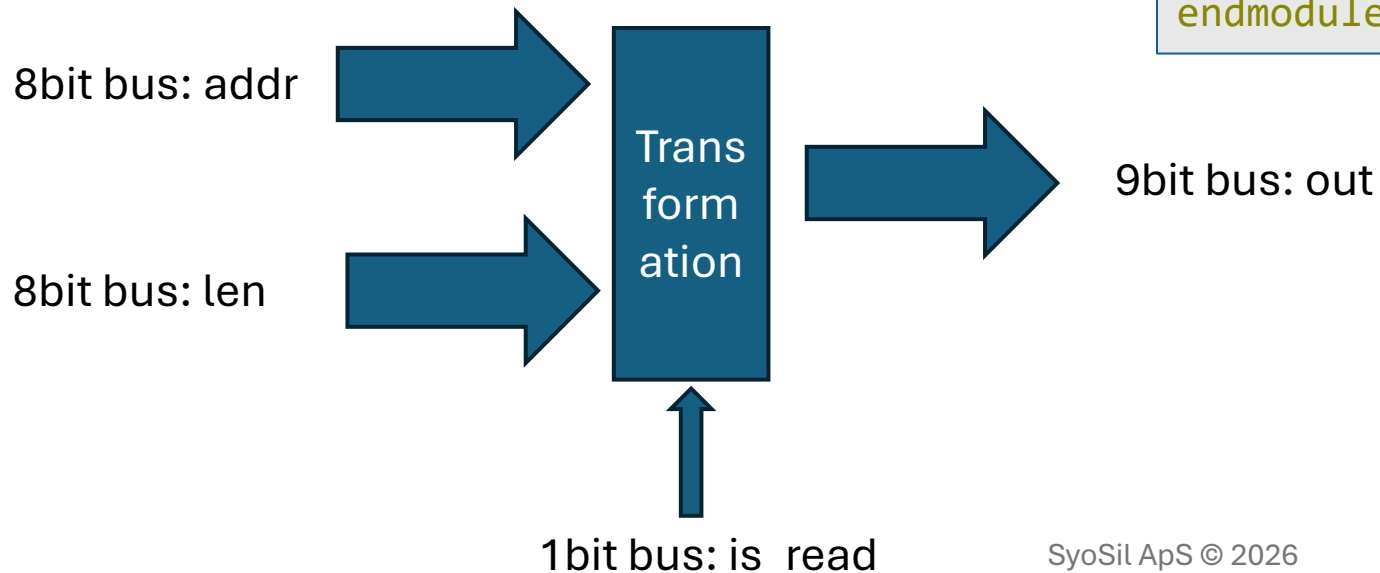
```
@vsc.randobj
class Packet:
    def __init__(self):
        self.addr      = vsc.rand_uint8_t() # 0..255
        self.length    = vsc.rand_uint8_t() # 0..255
        self.is_read    = vsc.rand_bit_t(1)  # 0/1
    ...
```

```
@vsc.randobj
class PacketCorner:
    def __init__(self):
        ...
        self.corner = vsc.rand_uint_t(2) # 0..3

    @vsc.constraint
    def corner(self):
        with vsc.if_then(self.coner == 0): # Corner0
            self.addr in vsc.rng(0, 15)
            self.length in vsc.rng(0, 15)
        with vsc.else_if(self.coner == 1): # Corner1
            self.addr in vsc.rng(240, 225)
            self.length in vsc.rng(0, 15)
    ...
```

What are We Verifying?

- What is important?
 - Individual bit toggle for each input vector
 - What about?
 - Addr=0x00, len=0x00
 - Addr=0x00, len=0xff
 - Addr=0xff, len=0x00
 - Addr=0xff, len=0xff
 - For this module it might be enough as we can understand the complete transformation at once.



```
module(input logic      clk,
       input logic      reset,
       input logic [7:0] addr,
       input logic [7:0] len,
       input logic      is_read,
       output logic [8:0] out);

always @(posedge clk) begin
    if (reset == 1) begin
        out <= `b0;
    else begin
        out <= (is_read == 0) ? addr+len : addr-len;
    end
end
endmodule
```

- Maybe some in between?
 - Merge state for $\text{addr} \% 2$ and $\text{len} \% 2$
 - 4 times less state
 - Still good coverage for the transformation